

1. Aggregation

- Aggregation represents a "HAS-A" relationship, where one class contains or uses an object of another class.

```
class Address:

    def __init__(self, city):
        self.city = city

class Student:

    def __init__(self, name, city):
        self.name = name
        self.address = Address(city)
```

Main Difference

Inheritance	Aggregation
IS-A relationship	HAS-A relationship
Child inherits parent	One object contains another object
class Car(Vehicle)	self.engine = Engine()
Promotes reuse through inheritance	Promotes reuse through composition/relationships

2. Method Overriding

- Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Polymorphism

- **Polymorphism** means the same method name can behave differently depending on the object.

```
class Animal:

    def sound(self):
        print("Animal sound")

class Dog(Animal):

    def sound(self):
        print("Dog barks")
```

3. Method Overloading

- Method overloading means defining multiple methods with the same name but with different parameters .
- Python does NOT support traditional method overloading like Java/C++.

Not Allow

```
class Calculator:

    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

Allow

```
class Calculator:

    def add(self, a, b, c=0):
        return a + b + c
```

4. Encapsulation

- Encapsulation is the process of combining data and the methods that operate on that data inside a class and controlling access to that data.

Why use encapsulation?

- Protect data
- Control access
- Validate data
- Prevent accidental modification
- Keep implementation details controlled

```
class BankAccount:

    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):

        if amount > 0:
            self.__balance += amount
```

5. Abstraction

- Abstraction means hiding unnecessary implementation details and showing only the essential functionality to the user.

When you use an ATM:

You see:

→ Withdraw

→ Deposit

→ Check Balance

You don't see:

- Internal banking server
- Database queries
- Transaction processing

```
class Vehicle(ABC):  
  
    def start(self):  
        pass  
  
class Car(Vehicle):  
  
    def start(self):  
        print("Car starts with key")
```

6. Instance Variable

- An instance variable belongs to a particular object, so different objects can have different values.

```
class Student:  
  
    def __init__(self, name):  
        self.name = name  
  
s1 = Student("Rahul")  
s2 = Student("Amit")
```

7. Class / Static Variable

- A class variable belongs to the class and is shared by instances unless an instance creates its own attribute with the same name.

```
class Student:

    college = "DBATU"

    def __init__(self, name):
        self.name = name
```

Instance vs Static Variable

Instance Variable	Class Variable
Belongs to object	Belongs to class
Different values possible	Common class-level value
Uses self	Defined directly in class
self.name	college
Created per object	Shared through class

8. Instance Method

- An instance method works with the data of a particular object and receives self as its first parameter.

```
class Student:

    def display(self):
        print(self.name)

s1 = Student()

s1.display()
```

9. Static Method

- A static method is a method that does not depend on instance data and does not receive self automatically.

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Three Types of Methods

Method	First Parameter	Works Mainly With
Instance Method	self	Object data
Static Method	None	Independent logic